package com.example.tastetoshare.screens import androidx.compose.foundation.background import androidx.compose.foundation.clickable import and…

PASTED

i built a popup inside this screen. it works but i want to fix it graphically. it almost fills the whole screen horizontally, while i would like to have it more squared. also, i'd like to have a bit of padding inside because the alert text is too close to the edges of the popup

Looking at your `UnsavePopup`, the issue is in the `Card`'s modifier — `padding(20.dp, 220.dp)` uses huge vertical padding which forces the card to stretch horizontally to fill the remaining space. You also need internal padding around the text.

Here's the fix:

kotlin

```kotlin
@Composable
fun UnsavePopup(
    unsaveRecipe: (String) -> Unit,
    hideUnsavePopUp: () -> Unit,
    recipeId: String
) {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Box(
            modifier = Modifier
                .fillMaxSize()
                .background(Color.Bla copy(alpha = 0.5f))
                .pointerInput(Unit) {
```

Scrivi un messaggio…

Sonnet 4.6

Claude è un'AI e può commettere errori. Verifica le risposte.

```
            .fillMaxSize()
            .background(Color.Black.copy(alpha = 0.5f))
            .pointerInput(Unit) {
                awaitPointerEventScope {
                    while (true) {
                        awaitPointerEvent()
                    }
                }
            }
    )

    Card(
        modifier = Modifier
            .padding(horizontal = 32.dp)  // controls width — increase to r
    ) {
        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center,
            modifier = Modifier.padding(24.dp)  // internal breathing room
        ) {
            Text(
                text = "Are you sure you want to remove this recipe from yo
            )
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(top = 20.dp),
                horizontalArrangement = Arrangement.SpaceEvenly
            ) {
                Button(
                    modifier = Modifier.weight(1F),
                    onClick = { hideUnsavePopUp() },
                ) {
                    Text("No")
                }
                Spacer(Modifier.w   h(16.dp))
                Button(
                    modifier = Modifier.weight(1F),
```

Scrivi un messaggio...

Sonnet 4.6

Claude è un'AI e può commettere errori. Verifica le risposte.

```
                    Text("No")
                }
                Spacer(Modifier.width(16.dp))
                Button(
                    modifier = Modifier.weight(1F),
                    colors = ButtonDefaults.buttonColors(
                        containerColor = DangerRed,
                        contentColor = MaterialTheme.colorScheme.onPrimary
                    ),
                    onClick = { unsaveRecipe(recipeId) },
                ) {
                    Text("Yes")
                }
            }
        }
    }
}
```

The two key changes:

- `padding(20.dp, 220.dp)` → `padding(horizontal = 32.dp)` — the original vertical padding of 220dp was what forced the card to expand sideways to compensate. Using only horizontal padding lets the card size naturally to its content height.
- `modifier = Modifier.padding(24.dp)` **on the** `Column` — replaces the `padding(top = 20.dp)` that only padded the top, giving even breathing room on all sides around the text and buttons.

```
data class Review @OptIn(ExperimentalUuidApi::class)
constructor(
    val reviewId: String = Uuid.random().toString(),
    val creationDate: Long = System.currentTimeMillis(),
    val photosUrl: List<String> = emptyList(),
    val body: String? = null,
```

Mostra altro

The `creationDate` is a Unix timestamp in **milliseconds**, so for May 9, 2025:

json

```
{
  "reviewId": "550e8400-e29b-41d4-a716-446655440000",
  "creationDate": 1746748800000,
  "photosUrl": [],
  "body": "Great product!",
  "rating": 5,
  "userId": "660e8400-e29b-41d4-a716-446655440001"
}
```

`1746748800000` = May 9, 2025 00:00:00 UTC

A quick way to get these values is epochconverter.com, just remember to pick **milliseconds** (multiply the standard Unix seconds by 1000).

```
colors = ButtonDefaults.buttonColors(
containerColor =
MaterialTheme.colorScheme.tertiaryContainer,
contentColor = MaterialTheme.colorScheme.onTertiaryContainer
),                    contentPadding = PaddingValues(0.dp)
) {                    Icon(
painter = painterResource(id = R.drawable.send),
contentDescription = "Enter review icon",
modifier = Modifier.size(20.dp)                    )
}            }         }     } }
```

this is the button to send a review. is it possible to make it round and make it float on top of the text field? now it's on its right and i don't like it

Mostra meno

Here's the updated composable with a floating round button overlaid on the bottom-right corner of the text field:

kotlin

```kotlin
@Composable
fun InputReviewCard(
    rating: UInt,
    reviewText: String,
    onRatingChanged: (Int) -> Unit,
    onReviewTextChanged: (String) -> Unit,
    onAddReview: () -> Unit,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier
            .width(300.dp)
            .heightIn(min = 210.dp),
        shape = RoundedCornerShape(25),
        colors = CardDefaults.cardColors(
```

Scrivi un messaggio...

Sonnet 4.6

Claude è un'AI e può commettere errori. Verifica le risposte.

```
) {
    Card(
        modifier = modifier
            .width(300.dp)
            .heightIn(min = 210.dp),
        shape = RoundedCornerShape(25.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.onPrimary
        ),
        elevation = CardDefaults.cardElevation(defaultElevation = 1.dp)
    ) {
        Column(
            modifier = Modifier
                .padding(16.dp)
                .fillMaxWidth()
        ) {
            Column(
                modifier = Modifier.fillMaxWidth(),
                verticalArrangement = Arrangement.spacedBy(8.dp),
            ) {
                Text(
                    text = "Share your opinion!",
                    style = MaterialTheme.typography.titleMedium.copy(fontSize
                    color = MaterialTheme.colorScheme.onSurface,
                    fontWeight = FontWeight.Bold
                )
                Spacer(modifier = Modifier.width(5.dp))
                Row {
                    for (i in 1..5) {
                        val icon = if (i.toUInt() <= rating) R.drawable.filled_
                        Icon(
                            painter = painterResource(id = icon),
                            contentDescription = "Star icon",
                            tint = StarGold,
                            modifier = Modifier
                                .siz .dp)
                                .clickable { onRatingChanged(i) }
```

Scrivi un messaggio...

Sonnet 4.6

Claude è un'AI e può commettere errori. Verifica le risposte.

```
                Icon(
                    painter = painterResource(id = icon),
                    contentDescription = "Star icon",
                    tint = StarGold,
                    modifier = Modifier
                        .size(32.dp)
                        .clickable { onRatingChanged(i) }
                )
            }
        }
    }

    Spacer(modifier = Modifier.height(8.dp))

    // Box lets the FAB float over the TextField
    Box(modifier = Modifier.fillMaxWidth()) {
        TextField(
            value = reviewText,
            onValueChange = onReviewTextChanged,
            modifier = Modifier.fillMaxWidth(),
            colors = TextFieldDefaults.colors(
                unfocusedContainerColor = MaterialTheme.colorScheme.onI
                focusedContainerColor = MaterialTheme.colorScheme.tert
            ),
            singleLine = false,
            minLines = 5
        )

        // Round floating button anchored to bottom-end of the Box
        SmallFloatingActionButton(
            onClick = onAddReview,
            modifier = Modifier
                .align(Alignment.BottomEnd)
                .offset(x = (-8).dp, y = (-8).dp), // inset from the c
            containerColor = MaterialTheme.colorScheme.tertiaryContaine
            contentColor = Ma ialTheme.colorScheme.onTertiaryContaine
            shape = CircleShape,
```

Scrivi un messaggio...

Sonnet 4.6

Claude è un'AI e può commettere errori. Verifica le risposte.

```
                    containerColor = MaterialTheme.colorScheme.tertiaryContaine
                    contentColor = MaterialTheme.colorScheme.onTertiaryContaine
                    shape = CircleShape,
                    elevation = FloatingActionButtonDefaults.elevation(defaultE
                ) {
                    Icon(
                        painter = painterResource(id = R.drawable.send),
                        contentDescription = "Send review",
                        modifier = Modifier.size(20.dp)
                    )
                }
            }
        }
    }
}
```

Key changes:

- Wrapped the `TextField` in a `Box` — this is what enables children to overlap each other using `align`.
- Replaced the `Button` with a `SmallFloatingActionButton` using `shape = CircleShape`, which gives you the round shape and a natural shadow/elevation out of the box.
- Used `Modifier.align(Alignment.BottomEnd)` to pin it to the bottom-right of the text field, with a small `offset` to nudge it slightly inward so it doesn't sit flush against the edge.
- The `Row` wrapping both components is gone since the button is now positioned independently via the `Box`.

You may want to add some `padding(bottom = ...)` to the `TextField` so the last line of text isn't hidden behind the button when the user types a lot.

Scrivi un messaggio...

Sonnet 4.6

Claude è un'AI e può commettere errori. Verifica le risposte.

data class Notification @OptIn(ExperimentalUuidApi::class)
constructor(
    val notificationId: String = Uuid.random().toString(),
    val creationDate: Long = System.currentTimeMillis(),
    val userId: String = "",
    val recipeId: String = "",
    val type: NotificationType =
NotificationType.RECOMMENDED_RECIPE,
    val title: String = "",
    val message: String = ""
)

can you create an hardcoded list of notifications (5) so i can use it for testing, these are the notification types:
enum class NotificationType(val displayName: String) {
    DUPLICATED_RECIPE("Duplicated recipe"),
    RECOMMENDED_RECIPE ("Recommended recipe"),
    NEW_REVIEW("New review");
}

Mostra meno

Sure — here's a hardcoded list of 5 sample `Notification` objects you can drop into your Kotlin code for testing:

Kotlin

```kotlin
val testNotifications = listOf(
    Notification(
        notificationId = "notif_001",
        creationDate = System.currentTimeMillis(),
        userId = "user_123",
        recipeId = "recipe_001",
        type = NotificationType.RECOMMENDED_RECIPE,
        title = "Recipe recommendation",
        message = "We think you'll love this creamy mushroom risotto!"
    ),
    Notification(
        notificationId = "notif_002",
        creationDate = System.currentTimeMillis() - 60000,
        userId = "user_123",
        recipeId = "recipe_002",
```

```
            type = NotificationType.NEW_REVIEW,
            title = "New review received",
            message = "Your lasagna recipe just received a 5-star review."
        ),
        Notification(
            notificationId = "notif_003",
            creationDate = System.currentTimeMillis() - 120000,
            userId = "user_456",
            recipeId = "recipe_003",
            type = NotificationType.DUPLICATED_RECIPE,
            title = "Duplicate recipe detected",
            message = "A similar carbonara recipe already exists in the system."
        ),
        Notification(
            notificationId = "notif_004",
            creationDate = System.currentTimeMillis() - 180000,
            userId = "user_789",
            recipeId = "recipe_004",
            type = NotificationType.RECOMMENDED_RECIPE,
            title = "New recommendation",
            message = "Check out this spicy ramen recipe picked for you."
        ),
        Notification(
            notificationId = "notif_005",
            creationDate = System.currentTimeMillis() - 240000,
            userId = "user_123",
            recipeId = "recipe_005",
            type = NotificationType.NEW_REVIEW,
            title = "Another review is in",
            message = "Someone commented on your chocolate cake recipe."
        )
    )
```

If you want more realistic testing, you could randomize `creationDate` or mix more `userId`s / `recipeId`s depending on your UI scenarios.

implement the paperDb queries needed for the following functions:

```
override fun getTopWeeklyRecipesPreview():
Flow<List<RawRecipePreview>> {
        //TODO compute one wekk ago date
// fitler recipes by creationDate >= one week ago
// sort by rating and then by number of rating
// map the recipe to RawRecipePreview
    }
```

```
    override fun getTopMonthlyRecipesPreview():
Flow<List<RawRecipePreview>> {
       //TODO compute one month ago date
// fitler recipes by creationDate >= one month ago
// sort by rating and then by number of rating
// map the recipe to RawRecipePreview    }
```

```
    override fun getFilteredTopWeeklyRecipesPreview(tags:
List<Tag>?): Flow<List<RawRecipePreview>> {
        //TODO compute one month ago date
// fitler recipes by creationDate >= one week ago
// fitler recipe, the recipe must contain at least one the of
the tags passed, if tags is empty return all recipes
// sort by rating and then by number of rating
// map the recipe to RawRecipePreview    }
    }
```

```
private val _recipes = MutableStateFlow<List<Recipe>>(
        Paper.book(bookName).allKeys.mapNotNull { key ->
            Paper.book(bookName).read<Recipe>(key)
        }
    )
```

the db contains Recipe:

```
data class Recipe @OptIn(ExperimentalUuidApi::class)
constructor(
    val recipeId: String = Uuid.random().toString(),
    val creationDate: Long = System.currentTimeMillis(),
    val originalRecipeId: String? = null,
    val authorId: String = "",
    val title: String = "",
    val description: String = "",
    val photoURL: String = "",
    val preparationTime: Int = 0,
    val servings : Int = 0,
    val rating : Float = 0F,
    val numberOfRatings : Long = -1,
    val difficulty : Difficulty = Difficulty.EASY,
    val cost: Cost = Cost.CHEAP,
    val calories: Int? = null,
    val ingredients : List<RecipeIngredient> = emptyList(),
    val steps : List<Step> = emptyList(),
    val tags: List<Tag> = emptyList(),
    val reviews: List<Review> = emptyList()
)
```

you should turn them into RawRecipePreview:

```
data class RawRecipePreview
@OptIn(ExperimentalUuidApi::class) constructor(
    val recipeId: String = Uuid.random().toString(),
    val creationDate: Long = System.currentTimeMillis(),
    val originalRecipeId: String? = null,
    val authorId: String = "",
    val title: String = "",
```

```
    val photoURL: String = "",
    val preparationTime: Int = 0,
    val rating : Float = 0F,
    val numberOfRatings : Long = -1,
    val difficulty : Difficulty = Difficulty.EASY,
    val cost: Cost = Cost.CHEAP,
    val calories: Int? = null,
    val ingredients : List<RecipeIngredient> = emptyList(),
    val tags: List<Tag> = emptyList(),
)
```

Mostra meno

You can implement these directly off `_recipes` by filtering the in-memory `StateFlow`, sorting, and mapping to `RawRecipePreview`.

A clean approach is to:

1. Create a mapper `Recipe -> RawRecipePreview`
2. Compute date cutoffs using `System.currentTimeMillis()`
3. Use `map {}` on `_recipes`

Kotlin

```kotlin
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import java.util.concurrent.TimeUnit

private fun Recipe.toRawRecipePreview() = RawRecipePreview(
    recipeId = recipeId,
    creationDate = creationDate,
    originalRecipeId = originalRecipeId,
    authorId = authorId,
    title = title,
    photoURL = photoURL,
    preparationTime = preparationTime,
    rating = rating,
    numberOfRatings = numberOfRatings,
    difficulty = difficulty,
    cost = cost,
    calories = calories,
    ingredients = ingredients,
    tags = tags
)

private fun sortRecipes(recipes: List<Recipe>): List<Recipe> =
    recipes.sortedWith(
```

```
            compareByDescending<Recipe> { it.rating }
                .thenByDescending { it.numberOfRatings }
        )

    override fun getTopWeeklyRecipesPreview(): Flow<List<RawRecipePreview>> {
        val oneWeekAgo = System.currentTimeMillis() - TimeUnit.DAYS.toMillis(7)

        return _recipes.map { recipes ->
            sortRecipes(
                recipes.filter { it.creationDate >= oneWeekAgo }
            ).map { it.toRawRecipePreview() }
        }
    }

    override fun getTopMonthlyRecipesPreview(): Flow<List<RawRecipePreview>> {
        val oneMonthAgo = System.currentTimeMillis() - TimeUnit.DAYS.toMillis(30)

        return _recipes.map { recipes ->
            sortRecipes(
                recipes.filter { it.creationDate >= oneMonthAgo }
            ).map { it.toRawRecipePreview() }
        }
    }

    override fun getFilteredTopWeeklyRecipesPreview(
        tags: List<Tag>?
    ): Flow<List<RawRecipePreview>> {
        val oneWeekAgo = System.currentTimeMillis() - TimeUnit.DAYS.toMillis(7)

        return _recipes.map { recipes ->
            sortRecipes(
                recipes.filter { recipe ->
                    val isRecent = recipe.creationDate >= oneWeekAgo
                    val matchesTags = tags.isNullOrEmpty() ||
                            recipe.tags.any { it in tags }

                    isRecent && matchesTags
                }
            ).map { it.toRawRecipePreview() }
        }
    }
```

Behavior

- **Weekly:** recipes created in last 7 days
- **Monthly:** recipes created in last 30 days
- **Filtered weekly:** last 7 days + contains **at least one** provided tag
- If `tags == null` or empty → returns all weekly recipes
- Sorting priority:
  1. Higher `rating`

2. Higher `numberOfRatings`

This keeps everything reactive since updates to `_recipes` automatically propagate through the returned `Flow`.